

Lesson 5: Forms, Controlled Inputs, and Sharing State with Context

Mission Brief

Ground Control needs a proper mission-request form — multiple fields, validation, and a submit action — and several unrelated parts of your app (a cart icon in the header, a cart drawer, a checkout summary) all need to see the *same* shopping data without passing it down through ten layers of props. Today you build a complete multi-field form, and learn `useContext` to share state across the component tree without "prop drilling."

Learning Objectives

By the end of this lesson, you will be able to:

- Build a multi-field controlled form with `useState`
- Validate form input and show inline error messages
- Handle form submission with `onSubmit` and `preventDefault`
- Explain the "prop drilling" problem and how Context solves it
- Create a Context, provide it, and consume it with `useContext`

What You'll Build

A `MissionRequestForm` with name, email, and message fields (with validation), plus a small `CartContext` that a `Header` badge and a `CartDrawer` both read from without passing props through `App`.

Prerequisites

- Comfortable with `useState` and event handlers (Lesson 3)
- Comfortable with `useEffect` (Lesson 4)

Key Concepts

- Controlled form fields for text, email, and textarea
- `event.preventDefault()` in `onSubmit`
- Simple validation and inline error display
- Prop drilling and why it becomes painful
- `createContext`, `<Context.Provider>`, and `useContext`

Concept Explanation

A form with several fields just means several pieces of state, one per field, each wired up the controlled-component way from Lesson 3:

```

function MissionRequestForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [message, setMessage] = useState("");
  const [errors, setErrors] = useState({});
  const [submitted, setSubmitted] = useState(false);

  function validate() {
    const next = {};
    if (!name.trim()) next.name = "Name is required.";
    if (!email.includes("@")) next.email = "Enter a valid email.";
    if (message.trim().length < 10) next.message = "Message must be at least 10 characters.";
    return next;
  }

  function handleSubmit(e) {
    e.preventDefault(); // stop the browser's default full-page reload
    const foundErrors = validate();
    setErrors(foundErrors);
    if (Object.keys(foundErrors).length === 0) {
      setSubmitted(true);
      // In a real app: send `{ name, email, message }` to a server here.
    }
  }

  if (submitted) return <p>✅ Request received, Commander {name}!</p>;

  return (
    <form onSubmit={handleSubmit} noValidate>
      <label>
        Name
        <input value={name} onChange={(e) => setName(e.target.value)} />
      </label>
      {errors.name && <p className="error">{errors.name}</p>}}

      <label>
        Email
        <input value={email} onChange={(e) => setEmail(e.target.value)} />
      </label>
      {errors.email && <p className="error">{errors.email}</p>}}

      <label>
        Message
        <textarea value={message} onChange={(e) => setMessage(e.target.value)} />
      </label>
      {errors.message && <p className="error">{errors.message}</p>}}

      <button type="submit">Send Request</button>
    </form>
  );
}

```

```
    </form>
  );
}
```

`e.preventDefault()` is essential — by default, submitting an HTML form reloads the entire page and sends its data to a URL, which is not what a React single-page app wants. Calling it lets you handle the submission entirely in JavaScript instead.

Now imagine `App` renders `Header`, which renders a cart icon with a badge showing the item count, and separately renders a `ProductInfo` component with an "Add to Cart" button, and a `CartDrawer` that lists items. All three need the *same* cart data. Passing it as props from `App` down through every intermediate component that doesn't otherwise need it — just so a distant grandchild can use it — is called **prop drilling**, and it gets unmanageable fast in a real app.

Context solves this by creating a piece of state that any descendant component can read directly, no matter how deep, without it being passed as a prop at every level:

```
// CartContext.jsx
import { createContext, useContext, useState } from "react";

const CartContext = createContext(null);

export function CartProvider({ children }) {
  const [items, setItems] = useState([]);

  function addItem(product) {
    setItems((prev) => [...prev, product]);
  }

  return (
    <CartContext.Provider value={{ items, addItem }}>
      {children}
    </CartContext.Provider>
  );
}

export function useCart() {
  return useContext(CartContext);
}
```

```
// App.jsx
function App() {
  return (
    <CartProvider>
      <Header />
      <ProductInfo />
      <CartDrawer />
    </CartProvider>
  );
}
```

```
// Header.jsx
function Header() {
  const { items } = useCart();
  return <span>🛒 {items.length}</span>;
}
```

`CartProvider` wraps the part of the tree that needs access and holds the actual state; any component inside it — no matter how deeply nested — calls `useCart()` and gets `{ items, addItem }` directly, with zero props passed in between. This is exactly the pattern behind Project 07's cart.

Context is best used for data that's genuinely global-ish to a section of your app (theme, logged-in user, a shopping cart) — not a replacement for regular props on components that only need data from their direct parent.

Guided Walkthrough

1. Build `MissionRequestForm` with the three fields, validation, and submit handling shown above; confirm submitting an empty form shows all three error messages and no page reload happens.
2. Fix the errors one at a time and confirm each error message disappears once that field becomes valid, and the success message appears after a fully valid submit.
3. Create `CartContext.jsx` exactly as shown, with `CartProvider` and the `useCart` hook.
4. Wrap your `App`'s returned JSX in `<CartProvider>` and create a `Header` component that displays `items.length` via `useCart()`.
5. Create a button elsewhere (e.g. in a `ProductInfo` component) that calls `addItem({ id: 1, name: "Rocket Model" })` from `useCart()`, and confirm the `Header` badge count updates — proving both components share one source of truth with no props passed between them.

Common Bugs & Debugging Tips

- Forgetting `e.preventDefault()` causes the page to visibly reload/flash on submit and lose all component state — always check this first if a form "resets everything" on submit.

- Calling `useContext(CartContext)` in a component that is *not* rendered inside `<CartProvider>` returns the default value passed to `createContext` (here `null`), causing a crash when you try to destructure `{ items }` from it — always double check the provider wraps the consumer.
- Validation that runs on every keystroke (instead of only on submit) can be jarring — this lesson's pattern only validates in `handleSubmit`, which is friendlier for beginners; more advanced apps validate `onBlur` too.
- Re-creating the context value object inline as a new object on every render (`value={{ items, addItem }}`) is fine for learning purposes but is a common source of unnecessary re-renders in larger apps — worth knowing as you grow, not a blocker now.

Check Your Understanding

1. Why is `e.preventDefault()` necessary inside a form's submit handler?
2. What problem does React Context solve that plain props don't?
3. What are the three things you need to use Context: what do you create, wrap, and call?
4. Why does `CartProvider` need to wrap `Header`, `ProductInfo`, and `CartDrawer` all inside one shared parent?
5. When would plain props be a better choice than Context?

Mini Quiz + Answer Key

- Q1.** What does `e.preventDefault()` stop from happening in a form's `onSubmit` handler? A) The function from running B) The browser's default full-page reload C) State updates D) Validation
- Q2.** What is "prop drilling"? A) A build tool feature B) Passing a prop through many intermediate components that don't need it, just to reach a deep descendant C) A type of CSS selector D) A React error message
- Q3.** What three pieces make up the Context pattern? A) `useState`, `useEffect`, `useRef` B) `createContext`, a `Provider` component, and `useContext` C) `fetch`, `.then`, `.catch` D) `props`, `state`, `key`
- Q4.** If a component using `useContext(CartContext)` isn't rendered inside `<CartProvider>`, what happens? A) It automatically creates its own provider B) It gets the default value passed to `createContext`, which can cause errors if code assumes real data C) React throws a compile error D) Nothing, it works fine

Answer Key: 1-B, 2-B, 3-B, 4-B

Lesson Recap

You built a fully validated, controlled multi-field form with proper submit handling, and learned how Context (`createContext`, a `Provider`, and `useContext`) lets far-apart components share one source of state without prop drilling — exactly the mechanism behind a real shopping cart.

Homework

Extend the cart context with a `removeItem(id)` function and an `total` value computed from item prices, then build a `CartSummary` component (rendered separately from `Header`) that displays the running total and a "Clear Cart" button, all via `useCart()` with no props passed in.

Portfolio Project Connection

Project 07's `CartContext` and `CartDrawer` , and Project 09's contact form, are built directly from this lesson's Context and controlled-form patterns.